Spectrum Dating

Product & Engineering Handoff Guide

A calm-by-design dating platform for autistic adults. This document is the complete package for handing the product to your team — written for both a **Product Manager** (what it is, what it does, what it needs to launch) and a **Senior Developer** (how it is built, hosted, and moved onto your own infrastructure and domain).

Status at handoff: Built, deployed, and functionally green (automated smoke suite 11/11, no product-breaking bugs).
Prepared: July 2026 · Final pre-handoff review
Scope: Full-stack monorepo (React front end + Node/Express back end), live on Vercel + Railway, ready to migrate to your accounts.

Contents

Sections 1–5 are written for a general/PM audience. Sections 6–7 are the technical handoff for your senior developer. Section 8 is the living engineering backlog.

1 · Executive summary

Spectrum Dating is complete and live in a staging/production environment. It works end-to-end today. What stands between it and a public launch is **not unbuilt software** — it is a short list of accounts, keys, vendor choices, and legal/business decisions that only you, the owner, can make.

Where it stands

- Full user journey works: sign-up → onboarding → discovery → matching → messaging.
- Automated quality gate passing (**11/11** smoke, 431 backend tests).
- Deployed: front end on Vercel, back end + database on Railway.
- A recent hardening pass fixed a batch of bugs and a real moderation gap (Section 4).

What it needs from you to launch

- **Config keys** (email, push, admin) — minutes each, Section 5A.
- **Vendor choices** — payments + trust-&-safety, Section 5D/5E.
- **Content** — the landing "who we are" copy, Section 5C.
- **Legal** — a lawyer's review of the Terms, Section 5G.

The one-sentence version

The product is built and safe to demo today; to open it to real users you need to plug in your own email/push/storage keys, choose a payment and a child-safety vendor, supply the landing copy, and have counsel review the Terms — all itemized in Section 5.

Handoff intent

The entire codebase (a single Git monorepo) is yours to take. It is designed to be re-hosted under your own Vercel + Railway (or equivalent) accounts and pointed at your own domain with no code changes — only environment configuration. Section 7 is the step-by-step migration guide.

2 · What Spectrum Dating is

A dating app built specifically for autistic adults, whose guiding principle is *calm by design*: it deliberately removes the anxiety-inducing, attention-extracting mechanics of mainstream dating apps.

Product principles ("product law")

These are hard rules enforced in code and in the automated tests. They are the brand's promise and should not be weakened without deliberate product sign-off:

- **No pressure mechanics.** No typing indicators, no read receipts, no "online/last seen", no streaks, no countdowns, no urgency, no gamification, no fabricated engagement metrics.
- **Honesty.** Nothing fabricated — no vanity "match %", no "X people viewed you", no scarcity nudges.
- **Sensory calm.** Reduced-sensory fallbacks for all decoration and motion; a low-stimulation mode; 13 selectable themes.
- **Privacy & safety first.** Coarse location only (city-level, never precise). Independent block and report. Identity (pride/trans/etc.) themes that reset on logout and can be reverted with a double-tap — a trust-&-safety feature for people in unsafe rooms.
- **Accessibility.** WCAG AA color contrast across every theme, full keyboard/screen-reader support, calm plain-language copy.

Who runs it

The admin/moderation console is itself built to be calm and accessible — the assumption is that moderators may also be autistic. It is not a dense enterprise dashboard.

3 · As-is functionality & feature inventory

Everything below is **built and working** in the deployed app today (unless explicitly marked as a scaffold awaiting your vendor choice).

Accounts & onboarding

- Email/password registration with bcrypt-hashed passwords and JWT sessions.
- Email verification & password reset flows (wired; need your email key — 5A).
- Guided, numbered onboarding: identity, preferences, coarse location, interests, and a **required first profile photo** (enforced server-side).
- Inclusive identity fields: expanded gender with an opt-out, orientation, pronouns, relationship goal/structure — display values kept separate from the 3-value matchable core used by matching.

Discovery & matching

- Scored candidate deck ("Discover") ranked by shared interests and compatibility, with hard filters (age, distance/radius, mutual gender-seeking) and calm swipe controls (interested / not now / skip).
- Mutual-like creates a match; a calm "match moment" (no confetti-casino energy).
- "Likes" / activity inbox (who liked you, recent matches) — no manipulative counts.
- Coarse-location distance only; precise location never leaves the server.

Messaging

- One-to-one conversations with a soft active-conversation cap (calm, not FOMO).
- Message requests (intro before full thread), attachments, and voice-note answers.
- Real-time delivery over authenticated WebSockets (socket.io), degrading gracefully.
- Archive, unmatch, block, and report — all reversible where appropriate.
- Deliberately no read receipts / typing indicators / "delivered" states.

Profile & personalization

- Rich profile: bio, prompts, structured "about me" facets, special interests, audio answers, photo gallery (approved-only public display).
- Profile hub with collapsible sections, notifications bell, completeness checklist.
- **13 themes** (dim default, light, navy, lightblue, pastel, pink + 7 LGBTQ+ identity flag themes) all meeting WCAG AA contrast; low-stimulation and reduced-motion modes.

Safety, moderation & trust

- Independent block & report from every relevant surface (card, profile, thread).
- A **Terms-of-Service-driven moderation console**: reports are auto-classified against the community standards and pre-filled with a suggested action, so admins act consistently instead of hand-writing each case (they can still add a personal reply, but the standards mean they rarely need to).
- Moderation queues for photos, audio, attachments, verification, reports, feedback — with test/demo accounts excluded so real work isn't buried in noise.

- Enforcement: warn / suspend / ban with due-process notices; banned/suspended members are removed from every discovery surface.
- In-app Terms of Service and Safety Center; identity-theme reset/revert.

Membership / billing scaffold — needs a provider

- A complete, provider-agnostic billing scaffold: entitlements engine, an admin "manual access" demo toggle, and a signature-verifying, idempotent webhook route.
- The paid "Companion" tier (advanced filters, a best-fits shortlist, audio answers) is fully demoable today; it needs a payment provider chosen and keyed to take real money (Section 5D).

Admin console

- Moderation queues, member management, enforcement actions, insights/telemetry, and the billing demo toggle — all behind an admin allowlist.

4 · Recent hardening pass — what was just fixed

Before this handoff, a multi-agent audit combed the whole product and a prioritized batch of fixes was shipped and live-verified. Highlights:

Area	Fix (all shipped & verified live)
Security / privacy	A block now fully severs profile visibility (photos, voice, coarse city) in both directions — previously a block didn't end the underlying match, so a blocked person could still fetch the profile. Added a regression test.
Moderation effectiveness	Suspended/banned members are now excluded from Discover *and* the activity inbox. Previously a ban only blocked login — the person still appeared as a live, swipeable card to everyone.
Messaging reliability	Fixed a desktop bug where a draft could bleed into the wrong conversation; messages now re-fetch on socket reconnect; retry can no longer double-send.
Onboarding	Fixed a dead-end where choosing "Prefer not to say" for gender blocked completion; it is now a real, inclusive, persistable choice.
Accessibility	Brought eyebrow-label contrast to WCAG AA across all themes; added focus rings to the safety-critical theme-revert control; fixed focus restoration and stacked-dialog Escape handling.
Admin	Media-review queue now counts pending audio (a mapper dropped it, showing a false all-clear); live counts now carry a live timestamp; demo-tier toggle reflects immediately.
Performance	Candidate-deck interest lookup went from an N+1 query to a single batched query (~28× on a large deck); logged-out visitors no longer download the whole app.
Trust & safety	You can now block/report the *person* from their profile (not just a voice note); reaction endpoint hardened; account deletion now erases the billing subscription row (GDPR).

One flagged "race condition" was investigated and found to be a false positive (the database layer is synchronous, so the sequence can't interleave) — it was documented rather than "fixed", to avoid adding needless complexity.

5 · What you must complete before launch

None of these can be finished from code alone — each needs an account, an API key, a vendor choice, an infra setting, content, or a legal/business decision that only you can make. BLOCKS LAUNCH DECIDE BEFORE REAL USERS CONFIG ONLY

5A · Config that turns on already-built features CONFIG

Set on Railway	Why it's yours	Effect until set
`ADMIN_EMAILS` BLOCK	The root-admin allowlist in your production environment; there's no first-run bootstrap and we can't write your secrets.	**The moderation console is unreachable and no photo can be approved — so no new user ever becomes visible.**
`RESEND_API_KEY` + `EMAIL_FROM` BLOCK	Needs your email-vendor (Resend) account and a verified sending domain (DNS).	Email verification *and* password reset are dead — a user who forgets their password is permanently locked out.
`VAPID_PUBLIC_KEY` + `VAPID_PRIVATE_KEY` + `VAPID_CONTACT_EMAIL` BLOCK	Web-push keys must live in your production environment (generate a VAPID keypair).	Push notifications return 503 — users get zero match/message pushes (all the subscribe UI works, silently).
`QA_BYPASS_SECRET` CONFIG	Optional. Lets automated QA skip the auth rate-limiter. Inert until set; never weakens the real limiter. **Never set in a public env.**	QA/automation may trip the auth limiter; no user impact.

5B · Reconnect the front-end deploy DECIDE

The Vercel↔GitHub webhook can intermittently lag/drop front-end deploys (this is a Vercel-side quirk; the back end is unaffected). Once it's under your Vercel account: clear any stuck builds and reconnect the Git integration in Vercel → Settings → Git.

5C · Content you must supply BLOCK

The landing page's "who we are" section is an explicit **placeholder** shown to every first-time visitor. Supply your real team/mission copy (we can drop it in or remove the section) — it's your brand voice, not ours.

5D · Payments — decisions before real revenue DECIDE

1. **Choose a provider** (Stripe / Paddle / …). Fees, tax, and merchant-of-record are business calls. The scaffold then needs that provider implemented against the existing interface, plus its API key + webhook signing secret + `BILLING_PROVIDER` on Railway.
2. **Reconcile the Companion feature list with reality.** The published catalog currently advertises some items that aren't built (higher photo cap, short-video answers, AI draft/tone help, relocation matching). Only advanced filters, the best-fits list, and audio answers actually ship. Before charging: trim the copy to what ships, or fund building the rest. Charging for unbuilt features breaks the "honest by design" brand.
3. **Finalize pricing.** The `$8.99/mo` / `$54/yr` figures are placeholders and there is no annual-plan selector yet — if you keep the annual claim, an annual price must actually exist.

5E · Trust-&-safety vendors DECIDE

Each needs an external account (and, for child-safety, legal registration in your name) — we can't sign up or transmit data on your behalf.

Need	Why it's yours	Priority
CSAM hash-match + NCMEC reporting (e.g. Thorn "Safer")	A legal obligation tied to your legal entity; NCMEC reporting requires your registration. Highest exposure (US law, UK Online Safety Act).	**Critical before public launch**
Unsolicited-image blur (ML vision)	Needs a hosted ML endpoint / vendor account.	High
Phone/SMS verification (e.g. Twilio)	Ban-evasion friction; needs an SMS vendor. (A reported-not-yet-actioned user can currently re-register with the same email.)	High
Selfie/photo verification (FaceTec/Veriff-class)	The "Verified" badge is a manual toggle today — either wire a privacy-light selfie match, or hide the badge until it exists.	Medium

5F · Operations decisions DECIDE

- **Photo-approval staffing.** Every new user is invisible until an admin approves their first photo — no auto-approve. At real signup volume this is a human bottleneck; plan a moderator rotation, or buy the auto-moderation vendor (5E) to auto-clear the safe majority.
- **Demo/test data in production.** ~500 demo profiles are intentionally live to cushion cold-start, and some founder test accounts with troll names are live. Before real users: purge test/troll accounts (we can run the admin wipe) and decide whether the demo personas stay.
- **Appeal channel.** Due-process copy points to `support@spectrum-dating.app` — confirm that inbox exists and is monitored, or ask us to wire an in-app appeal form instead.

5G · Legal BLOCK

We drafted a plain-language, mission-transparent Terms of Service (in the repo and in-app). It is **not legal advice.** Your counsel must review it — especially the minors/NCMEC, liability/arbitration/governing-law, and data-retention clauses — before it's your binding, published Terms.

5H · Infra hardening CONFIG

- Set an object-size ceiling on the R2 media bucket (the app caps ≤5 MB pre-sign; the bucket cap is the real backstop).
- Add an R2 lifecycle rule to expire orphaned uploads.
- Optional: a dedicated private bucket for voice notes (higher PII).

6 · Senior developer handoff

Architecture, hosting, external services, and configuration — everything needed to run, operate, and take ownership of the system.

6.1 Architecture at a glance

Two deployable units in one Git monorepo:

- **Front end** — a React 18 single-page app built with Vite. Static assets served by **Vercel**. Talks to the back end over HTTPS + a WebSocket.
- **Back end** — a Node.js/Express API with a socket.io real-time layer and an embedded SQLite database, hosted on **Railway**. Serves a JSON API and issues JWTs; stores media in Cloudflare R2 object storage.

6.2 Repository layout (monorepo)

Path	What it is	Deploys to
`/` (root: `src/`, `index.html`)	Vite front end (React). Root directory `.`	Vercel
`/server`	Express + socket.io API, own `package.json`, own ESLint + Vitest, SQLite migrations. Root directory `server`	Railway
`/server/src/migrations`	60 sequential SQL migrations (run on boot)	—
`/scripts/qa`	End-to-end QA harness + smoke suite (Playwright-driven)	—
`/audit/handoff`	This guide and its source documents	—

6.3 Technology stack

Front end

React 18 · React-DOM · Vite (build) · socket.io-client · inline styles from a design-token file mapped to CSS variables · ESLint (react-hooks rules-of-hooks enforced) · no CSS framework, no state library.

Back end

Node ≥20 · Express · socket.io · JWT (`jsonwebtoken`) · bcrypt · **better-sqlite3** (embedded, synchronous) · helmet · cors · express-rate-limit · Vitest (431 tests).

6.4 External services & APIs the site depends on

Service	Used for	Library / access	Config keys
Cloudflare R2 (object storage)	Profile photos, voice notes, attachments. Accessed via the S3 API with presigned upload/download URLs.	`@aws-sdk/client-s3` + `s3-request-presigner`	`R2_ACCOUNT_ID`, `R2_ACCESS_KEY_ID`, `R2_SECRET_ACCESS_KEY`, `R2_BUCKET_NAME`, `R2_PUBLIC_URL`

Resend (email)	Verification, password reset, transactional email.	`resend`	`RESEND_API_KEY`, `EMAIL_FROM`
Web Push (VAPID)	Match/message push notifications to browsers.	`web-push`	`VAPID_PUBLIC_KEY`, `VAPID_PRIVATE_KEY`, `VAPID_CONTACT_EMAIL`
socket.io (real-time)	Live message/match delivery. JWT-authenticated on the handshake; auto-rejects on suspend/ban/token-bump.	`socket.io` / `-client`	(shares JWT)
geoip-lite	Coarse (city-level) location from IP — bundled, no external call.	`geoip-lite`	—
Payment provider	Not yet chosen — scaffold ready (Section 5D).	pluggable interface	`BILLING_PROVIDER` + provider keys

The front end reaches the back end through a single variable, `VITE_API_URL`, baked in at build time. There are no other third-party scripts or trackers on the front end.

6.5 Data & storage

- **Database:** SQLite via `better-sqlite3`, a single file on the Railway volume (`DB_PATH`). 60 sequential migrations run at boot. Synchronous access (simplifies concurrency reasoning). Suitable for launch scale; a move to Postgres is a known future step if volume demands it.
- **Media:** never stored in the DB — only an R2 object key is kept; bytes live in Cloudflare R2 behind unguessable keys, served via presigned URLs.
- **Backups:** an optional secondary bucket key exists (`R2_BACKUP_BUCKET`); wire your own DB volume backup cadence on Railway.

6.6 Deploy pipeline (current)

- **Front end (Vercel):** auto-deploys the `master` branch on push (~35 s). Build: `vite build` with `VITE_API_URL` set in the environment. Output is static; no server functions.
- **Back end (Railway):** deployed via the Railway CLI (`server/scripts/deploy.mjs` runs `railway up` and then blocks until `/health` reports the new commit SHA — so a broken deploy can't pass silently). Root directory is `server`; start command `node src/index.js`.
- **Health check:** `GET /health` returns `{ status, sha, db }` — the deployed Git SHA and DB status.
- **Quality gates:** front-end ESLint (0 errors, rules-of-hooks enforced), a Playwright smoke suite (`scripts/qa/smoke.mjs`, 11 checks), and back-end Vitest (431 tests).

6.7 Live environment (to be transferred to you)

Front-end URL	`spectrum-dating-eta.vercel.app` (Vercel; will become your domain)
Back-end URL	`spectrum-dating-server-production.up.railway.app` (Railway)
Front-end host	Vercel — static SPA, root directory `.`
Back-end host	Railway — Node service, root directory `server`, plus a persistent volume for the SQLite DB

Object storage	Cloudflare R2 bucket (default `spectrum-dating-photos`)

7 · Moving it to your own accounts & domain

The system is designed to move to your infrastructure with configuration only — no code changes. Recommended order:

1. **Take the repository.** Transfer or fork the Git monorepo into your own GitHub organization.
2. **Stand up storage.** Create a Cloudflare R2 bucket (+ a public URL / custom domain for it) and generate an access key pair. Note the six `R2_*` values.
3. **Stand up the back end (Railway or equivalent).** New project → root directory `server` → attach a persistent volume for the SQLite file (`DB_PATH`) → set all back-end environment variables (Section 6.4 + 5A) → deploy. Migrations run automatically on first boot. Confirm `/health`.
4. **Stand up the front end (Vercel or equivalent).** Import the repo → root directory `.` → set `VITE_API_URL` to your back-end URL → deploy.
5. **Point your domain.** Add your custom domain in Vercel and update the back end's allowed-origin / public-origin variables to match. Update `APP_URL` so email links use your domain.
6. **Turn on services.** Add the Resend key + verified domain, generate and add VAPID keys, set `ADMIN_EMAILS`, and (when ready) the payment + T&S vendor keys.
7. **Seed / clean data.** Decide on the demo profiles and purge test/troll accounts (Section 5F).

> **What does *not* need to change**
> Application code, database schema/migrations, the billing interface, feature gating, and the front-end build. Everything environment-specific is an environment variable. The only build-time value the front end needs is `VITE_API_URL`.

Local development quick start

- Back end: `cd server && npm install && npm run dev` (needs the env vars; SQLite auto-creates).
- Front end: `npm install && VITE_API_URL=http://localhost:<port> npm run dev`.
- Tests: back end `cd server && npm test` (Vitest); front-end lint `npx eslint .`; E2E smoke `node scripts/qa/smoke.mjs`.

8 · Remaining backlog (tracked, non-blocking)

None of these block launch. They are the polish/robustness items left after the hardening pass — recorded so your team can prioritize them. A nightly automated audit keeps this list refreshed.

Robustness & correctness

- Password re-entry on account deletion (a hijacked session can currently delete an account).
- Ban-evasion ledger (salted email/device hash checked at registration) — pairs with the SMS vendor (5E).
- Quarantine-on-reject for media instead of immediate delete (preserves CSAM evidence).
- Orphaned-attachment cleanup on account deletion; drop a legacy token-in-query-string fallback on the export endpoint.
- Add direct reporter reasons for off-platform harm and minor-safety; add pagination to the admin reports/audit-log endpoints.
- Name/pronoun screening extension (troll input passes today) — pairs with the test-data purge (5F).

Product & UX polish

- Trim the Companion catalog copy to shipped features (or build the rest) — Section 5D.
- Add an annual-plan selector if the annual price stays; rename "Top Picks" copy.
- Orientation opt-out parity with gender; clearer labels on the profile hub's icon buttons.
- Friendlier empty-Discover and pre-photo onboarding states; reframe the completeness checklist away from a "grade".

Engineering & tech-debt

- Introduce a schema-migrations ledger (migrations currently re-run idempotently each boot).
- Subset the display serif font; key admin tables by id rather than index.
- Refresh/retire stale status docs and the small pool of stale QA driver assertions.
- Longer term: evaluate SQLite → Postgres when volume warrants.

Companion source documents (in `/audit/handoff`, also delivered separately): `CLIENT_ACTIONS.md` (the client to-do list), `BACKLOG.md` (the full severity-ranked engineering backlog with exact file:line references), and `HANDOFF_SUMMARY.md` (a one-page status). This guide consolidates and supersedes them for handoff.